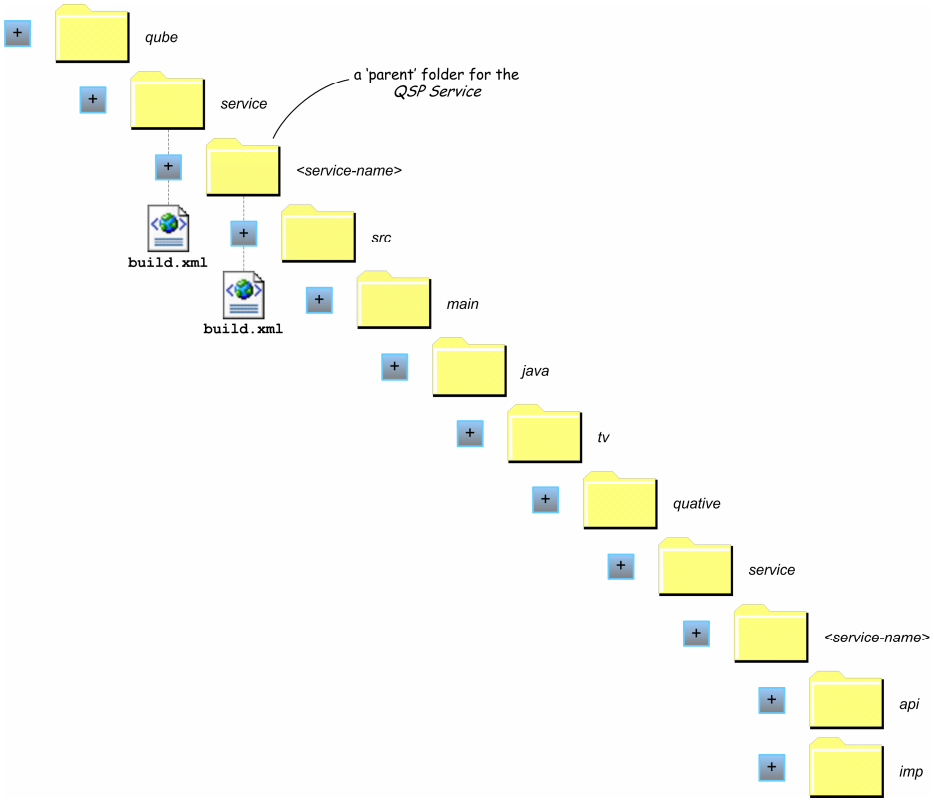
2. Overview of simple *Services*

A simple *Service* is the implementation of some functionality as a *POJO* that is accessed through the *Qube Service Gateway*. To expose these *Services* – we annotate the *Classes* and *Methods* with the annotations *@QubeService* (for *Classes*) and *@QubeMethod* (for *Methods*). We only need to annotate the *Classes* and *Methods* that we mean to expose to the *Qube Service Gateway*.

The following illustration shows the *Java* files required to implement a simple *Service*.

2.1. The *Service* developing file-structure

The following illustration shows the generic *Service* developing file-structure.



2.1.1. The *<service-name>.java* source file

The *<service-name>.java* source file 'lives' in the *api* folder shown in the illustration above and defines the *Interface* for the *Service*.

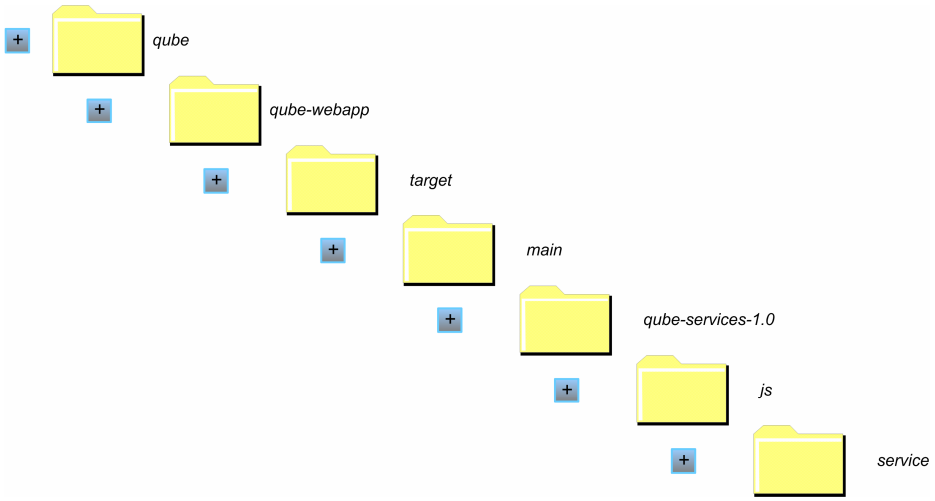
2.1.2. The *<service-name>Impl.java* source file

The *<service-name>Impl.java* source file 'lives' in the *imp* folder shown in the illustration above and defines the *Implementation* of the *Service*. In this file *Classes* and *Methods* are exposed to the *Qube*

Service Gateway with the annotations *@QubeService* and *@QubeMethod*. This file contains all the *Business Logic* of the *Service*.

2.1.3. The *Build* system

Although the *Build* system is beyond the scope of this document – the *build.xml* files shown in the illustration above (and discussed below) are essential to it. One of the tasks – essential for this *Tutorial* – the *Build* system carries out is generating *JavaScript* files for your *Services*. The following illustration shows the folder sub-structure that contains these *JavaScript* files.



The JavaScript file in the service folder are named following the convention:

<service-name>Service.js

The *JavaScript* files are used when your *Service* is called over *HTTP*.

3. Implementing the *BeanJuice Service*

The simple *BeanJuice Service* takes a single parameter (your name) and returns a message with your name appended.

The *Build* process requires the two *XML* build files shown in the illustration above.

3.1. The *\qube\service\build.xml* file

The following extract is an edited *\qube\service\build.xml* file that shows the reference to the 'parent' folder of the *Service*.

```
  …
  <filelist id="sub.projects" dir="${basedir}">
    …
    <file name="example-service-beanjuice"/>
    …
  </filelist>
</project>
```

3.2. The *\qube\service\example-service-beanjuice\build.xml* file

The *\qube\service\example-service-beanjuice\build.xml* file must be customised for each *Service*. The following extract shows the two places where this file is customised for this *Service*.

```
<project name="beanjuice-service" default="build" basedir=".">
  …
  <property name="name" value="service-beanjuice"/>
  …
</project>
```

3.2.1. The *BeanJuiceService.java* source file

The following extract is the *BeanJuiceService.java* source file that defines the *Interface* for the *BeanJuice Service*.

```
/**
 * @(#)BeanJuiceService.java
 *
 * Copyright (c)2007 Quative Ltd. All Rights Reserved.
 */
package tv.quative.service.beanjuice.api;

import tv.quative.service.Service;

 /**
 * An example of a simple Service.
 * @author Joe Dorward
 */
public interface BeanJuiceService extends Service {
 /**
  * Returns the 'Bean Juice' message.
  * @param name - the name of the User
  * @return String - the 'Bean Juice' message and the User name
  * @throws Exception - on error
  */
  String beanJuiceMessage(String name) throws Exception;
}
```

3.2.2. The *BeanJuiceServiceImpl.java* source file

The following extract is the *BeanJuiceServiceImpl.java* source file that defines the *Implementation* of the *BeanJuice Service*.

```
/**
 * @(#)BeanJuiceServiceImpl.java
 *
 * Copyright (c)2007 Quative Ltd. All Rights Reserved.
 */
package tv.quative.service.beanjuice.impl;

import tv.quative.service.BaseService;
import tv.quative.service.QubeMethod;
import tv.quative.service.QubeService;

import tv.quative.service.beanjuice.api.BeanJuiceService;

 /**
  * An example of a simple Service.
  * @author Joe Dorward
  * @version 1.0
  */
@QubeService(name = "beanJuiceService", ws = true, hue = true)
public class BeanJuiceServiceImpl extends BaseService implements BeanJuiceService {
 /**
  * Returns the 'Bean Juice' message.
  * @param name - the name of the User
  * @return String - the 'Bean Juice' message and the User name
```

```
  * @throws Exception - on error
  */
  @QubeMethod (ws = true, hue = true, roles = "all")
  public String beanJuiceMessage(final String name) throws Exception {
    return "Bring me the Bean Juice! " + name;
  }
}
```

By convention the name of a *Class* implemented as a *Service* is named following the form:

<service-name>ServiceImpl

In the above extract (the *@QubeService* annotation):

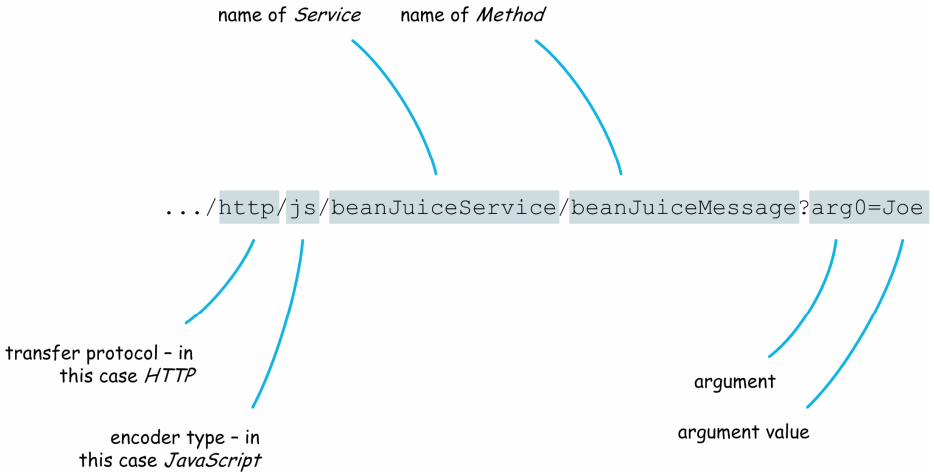
- The *@QubeService* annotation designates the *Class* as a *Service*
- The string value *beanJuiceService* of the *name* attribute is the name of the *Service*
- The string value *true* of the *ws* attribute means the *Service* will be accessible using *Web Services* – *false* would make it inaccessible
- The string value *true* of the *hue* attribute means the *Service* will be accessible using *HTTP* – *false* would make it inaccessible

3.3. Building and deploying the *Service*

Although building and deploying *Services* is beyond the scope of this document – the next step assumes you've done that.

3.4. Calling the *BeanJuice Service* over *HTTP*

Because the *BeanJuice Service* is exposed to the *HUE Gateway Interface* – you can test it using a standard web browser by typing a *URL* into its *Address Box*. The following illustration shows how to call the *BeanJuice Service* this way.

The above extract shows the *URL* used to call the *BeanJuice Service* and the result of that call – a *JavaScript* object – in the web browser.

The following illustration explains the component pieces of the *URL* shown above.